

HA 5 Prep

Organisatorisches

- Übung nächste Woche verschoben auf den Donnerstag, 18.06. 13:45 - 17:00 und findet online statt
- In 2 Wochen am Mittwoch, 24.06., wie gewohnt Deadline für mündliche Kontrollen der 5. HA, allerdings kein neuer Stoff
- In 3 Wochen am Mittwoch, 01.07. machen wir die Klausurvorbereitung
- In 4 Wochen am Mittwoch, 08.07., Abnahme für 6. HA

Design Patterns

Builder

- Usecase: Viele (optionale) Parameter im Konstruktor

```
public class ComplexBowl {
    private final Ingredient base;
    private final Ingredient protein;
    private final Ingredient sauce;    // Required: at least one sauce
    private final Ingredient topping;
    private final Ingredient topping2;
    private final Ingredient topping3;

    // Constructor
    public ComplexBowl(Ingredient base, Ingredient protein,
                      Ingredient sauce, Ingredient topping,
                      Ingredient topping2, Ingredient topping3) {
        this.base = base;
        this.protein = protein;
        this.sauce = sauce;
        this.topping = topping;
        this.topping2 = topping2;
        this.topping3 = topping3;
    }
}
```

Builder

- Builder-Klasse
- add Funktion, welche eine Instanz der Builderklasse zurückgibt
- build Funktion, welche das gewünschte Objekt “baut” und zurückgibt

Singleton

- Usecase: Eine Klasse soll genau eine Instanz haben und globalen Zugriff bereitstellen

Singleton

- Konstruktor muss private sein → keine andere Klasse kann es aufrufen
- Private static Variable, welche die einzige Instanz speichert
- public getter Methode, um auf die globale Instanz zugreifen zu können

Singleton

```
public class RecycleBin {  
  
    // 1. The single, private instance of the Recycle Bin  
    private static RecycleBin instance;  
  
    private List<String> deletedFiles;  
  
    // 2. Private constructor to prevent the creation of a new bin from outside  
    private RecycleBin() {  
        this.deletedFiles = new ArrayList<>();  
    }  
  
    // 3. Public accessor method (Lazy Initialization)  
    public static RecycleBin getInstance() {  
        if (instance == null) {  
            instance = new RecycleBin();  
        }  
        return instance;  
    }  
}
```


Observer

- Usecase: Benachrichtigung von Objekten bei Änderungen

Observer

- **Observable:**
 - hat eine Sammlung von Observern
 - kann Observer hinzufügen/entfernen
 - benachrichtigt alle Observern, wenn sich etwas ändert
- **Observer**
 - wird benachrichtigt, wenn es Änderung gibt

Spring Boot/Shell

Annotations

- `@Override`: Überschreiben von Funktionen einer Elternklasse oder einem Interface
- `@Test`: Markiert eine Funktion als eine Testfunktion für JUnit

Annotations

- @ShellComponent: Die Klasse enthält Shell Befehle
- @ShellMethod: Markiert eine Funktion als Shell Befehl
- @ShellOption: Customizing von Parametern

Annotations

- @Service: Spring erstellt automatisch eine Instanz für dieses Objekt
- @Primary: Setze Präferenz auf diese Klasse